

```
//Tarea.cs
using System;using System.Globalization;
namespace GestorTareas
{
    /// <summary>
    /// Representa el estado en el que puede encontrarse una tarea.
    /// </summary>
    public enum EstadoTarea
    {
        Pendiente,
        Completada
    }

    /// <summary>
    /// Clase que modela una tarea del gestor de pendientes.
    /// Aplica encapsulamiento mediante propiedades con validación básica.
    /// </summary>
    public class Tarea
    {
        // Formato usado para persistir y mostrar fechas de forma consistente.
        public const string FormatoFecha = "yyyy-MM-dd";

        // Separador usado al guardar en el archivo de texto (CSV simple).
        public const char Separador = ',';

        public int Id { get; set; }
        public string Titulo { get; set; }
        public string Descripcion { get; set; }
        public DateTime Fecha { get; set; }
        public EstadoTarea Estado { get; set; }

        public Tarea(int id, string titulo, string descripcion, DateTime fecha, EstadoTarea estado =
EstadoTarea.Pendiente)
        {
            Id = id;
            Titulo = titulo;
            Descripcion = descripcion;
            Fecha = fecha;
            Estado = estado;
        }

        /// <summary>
        /// Convierte la tarea a una línea de texto lista para escribir en el archivo.
        /// Se reemplaza el separador dentro de los campos para no romper el formato.
        /// </summary>
        public string ToLineaArchivo()
        {
            string tituloSeguro = Titulo.Replace(Separador, ',');
            string descripcionSeguro = Descripcion.Replace(Separador, ',');
            return string.Join(Separador,
                Id.ToString(CultureInfo.InvariantCulture),
                tituloSeguro,
                descripcionSeguro,
                Fecha.ToString(FormatoFecha, CultureInfo.InvariantCulture),
                Estado.ToString());
        }

        /// <summary>
        /// Reconstruye una Tarea a partir de una línea leída del archivo.
        /// Lanza FormatException si la línea está corrupta o incompleta.
        /// </summary>
        public static Tarea FromLineaArchivo(string linea)
        {
            string[] partes = linea.Split(Separador);
            if (partes.Length != 5)
            {

```

```

        throw new FormatException($"La línea no tiene el formato esperado (5 campos): '{linea}'");
    }

    int id = int.Parse(partes[0], CultureInfo.InvariantCulture);
    string titulo = partes[1];
    string descripcion = partes[2];
    DateTime fecha = DateTime.ParseExact(partes[3], FormatoFecha, CultureInfo.InvariantCulture);
    EstadoTarea estado = (EstadoTarea)Enum.Parse(typeof(EstadoTarea), partes[4]);

    return new Tarea(id, titulo, descripcion, fecha, estado);
}

public override string ToString()
{
    string estadoTexto = Estado == EstadoTarea.Completada ? "Completada" : "Pendiente";
    return $"[{Id}] {Titulo} | {Descripcion} | Fecha: {Fecha.ToString(FormatoFecha)} | Estado: {estadoTexto}";
}
}
}

```

//TareaNoEncontradaExeption.cs

```

using System;
namespace GestorTareas
{
    /// <summary>
    /// Excepción personalizada que se lanza cuando se intenta operar
    /// sobre una tarea que no existe en la colección (por Id).
    /// </summary>
    public class TareaNoEncontradaException : Exception
    {
        public int IdBuscado { get; }

        public TareaNoEncontradaException(int idBuscado)
            : base($"No se encontró ninguna tarea con el Id {idBuscado}.")
        {
            IdBuscado = idBuscado;
        }

        public TareaNoEncontradaException(int idBuscado, string mensaje)
            : base(mensaje)
        {
            IdBuscado = idBuscado;
        }
    }
}

```

//TareaRepositorio.cs

```

using System;using System.Collections.Generic;using System.IO;
namespace GestorTareas
{

```

```

/// <summary>
/// Encapsula la lectura y escritura de tareas en un archivo de texto (.csv),
/// usando StreamReader / StreamWriter, con manejo de errores de E/S.
/// </summary>
public class TareaRepositorio
{
    private readonly string _rutaArchivo;

    public TareaRepositorio(string rutaArchivo)
    {
        _rutaArchivo = rutaArchivo;
    }

    /// <summary>
    /// Carga todas las tareas desde el archivo. Si el archivo no existe,
    /// retorna una lista vacía (primer uso del programa).
    /// Las líneas corruptas se omiten y se reportan, sin detener la carga.
    /// </summary>
    public List<Tarea> Cargar()
    {
        var tareas = new List<Tarea>();

        if (!File.Exists(_rutaArchivo))
        {
            return tareas; // No hay datos previos todavía.
        }

        StreamReader? lector = null;
        try
        {
            lector = new StreamReader(_rutaArchivo);
            string? linea;
            int numeroLinea = 0;

            while ((linea = lector.ReadLine()) != null)
            {
                numeroLinea++;
                if (string.IsNullOrEmpty(linea)) continue;

                try
                {
                    tareas.Add(Tarea.FromLineaArchivo(linea));
                }
                catch (FormatException ex)
                {
                    Console.WriteLine($"Aviso: se omitió la línea {numeroLinea} del archivo por formato inválido
({ex.Message}).");
                }
            }
        }
        catch (IOException ex)
        {
            Console.WriteLine($"Error de E/S al leer el archivo de tareas: {ex.Message}");
        }
        catch (UnauthorizedAccessException ex)
        {
            Console.WriteLine($"No se tienen permisos para leer el archivo de tareas: {ex.Message}");
        }
        finally
        {
            lector?.Close();
        }

        return tareas;
    }

    /// <summary>

```

```

/// Guarda (sobrescribe) todas las tareas en el archivo.
/// Se llama luego de cada operación de agregar, completar o eliminar.
/// </summary>
public void Guardar(List<Tarea> tareas)
{
    StreamWriter? escritor = null;
    try
    {
        escritor = new StreamWriter(_rutaArchivo, append: false);
        foreach (Tarea tarea in tareas)
        {
            escritor.WriteLine(tarea.ToLineaArchivo());
        }
    }
    catch (IOException ex)
    {
        Console.WriteLine($"Error de E/S al guardar el archivo de tareas: {ex.Message}");
    }
    catch (UnauthorizedAccessException ex)
    {
        Console.WriteLine($"No se tienen permisos para escribir el archivo de tareas: {ex.Message}");
    }
    finally
    {
        escritor?.Close();
    }
}
}
}

```

//PersistenciaDatos

```

using System;using System.Collections.Generic;using System.IO;
public class PersistenciaDatos
{
    // Ruta del archivo donde se guardarán las tareas
    private readonly string _rutaArchivo;

    public PersistenciaDatos(string nombreArchivo = "tareas_guardadas.txt")
    {
        // Obtiene la ruta donde se esta ejecutando el programa
        string directorioEjecucion = AppDomain.CurrentDomain.BaseDirectory;

        // Sube 3 niveles en las carpetas para llegar a la raiz del proyecto
        string directorioRaiz = Path.GetFullPath(Path.Combine(directorioEjecucion, @"..\..\"));

        // Une la ruta de la raiz con el nombre del archivo
        _rutaArchivo = Path.Combine(directorioRaiz, nombreArchivo);
    }

    // Guarda la lista completa de las tareas en un archivo de texto
    // sobrescribiendo el archivo con la informacion más reciente
    public void GuardarTareas(IEnumerable<Tarea> tareas)
    {
        try
        {
            // El false indica que sobrescribirá el archivo
            using (StreamWriter sw = new StreamWriter(_rutaArchivo, false))
            {
                foreach (var tarea in tareas)
                {

```

```

        // Guardamos el Nombre y el estado de Completada
        string linea = $"{tarea.Nombre} {tarea.Completada}";
        sw.WriteLine(linea);
    }
}
}
catch (Exception ex)
{
    Console.WriteLine($"Error al guardar las tareas: {ex.Message}");
}
}

// Lee el archivo de texto y devuelve la lista de tareas guardadas
public List<Tarea> CargarTareas()
{
    List<Tarea> tareasCargadas = new List<Tarea>();

    // Si el archivo no existe, se devuelve una lista vacia
    if (!File.Exists(_rutaArchivo))
    {
        return tareasCargadas;
    }

    try
    {
        using (StreamReader sr = new StreamReader(_rutaArchivo))
        {
            string linea;
            while ((linea = sr.ReadLine()) != null)
            {
                string[] datos = linea.Split('|');

                // Se revisa que se tengan los 2 campos requeridos (Nombre y Completada)
                if (datos.Length == 2)
                {
                    Tarea tarea = new Tarea(datos[0])
                    {
                        Completada = bool.Parse(datos[1])
                    };

                    tareasCargadas.Add(tarea);
                }
            }
        }
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Error cargando las tareas: {ex.Message}");
    }

    return tareasCargadas;
}
}

```

//GestorTareas.cs

```

using System; using System.Collections.Generic;
public class Tarea
{
    public string Nombre { get; set; }
    public bool Completada { get; set; }
}

```

```

public Tarea(string nombre)
{
    Nombre = nombre;
    Completada = false;
}
}
class Program
{
    static List<Tarea> tareas = new List<Tarea>();
    // Necesario para la clase PersistenciaDatos.cs
    static PersistenciaDatos persistencia = new PersistenciaDatos();

    static void Main(string[] args)
    {
        int opcion;

        // Necesario para cargar los Datos en la lista guardada (.txt)
        tareas = persistencia.CargarTareas();

        do
        {
            Console.Clear();

            Console.WriteLine("=====");
            Console.WriteLine("    GESTOR DE TAREAS");
            Console.WriteLine("=====");
            Console.WriteLine("1. Agregar tarea");
            Console.WriteLine("2. Listar tareas");
            Console.WriteLine("3. Completar tarea");
            Console.WriteLine("4. Eliminar tarea");
            Console.WriteLine("5. Guardar lista de tareas");
            Console.WriteLine("6. Salir");
            Console.WriteLine("=====");
            Console.Write("Seleccione una opción: ");

            if (int.TryParse(Console.ReadLine(), out opcion))
            {
                switch (opcion)
                {
                    case 1:
                        AgregarTarea();
                        break;

                    case 2:
                        ListarTareas();
                        break;

                    case 3:
                        CompletarTarea();
                        break;

                    case 4:
                        EliminarTarea();
                        break;

                    case 5:
                        GuardarListaTareas();
                        break;

                    case 6:
                        Console.WriteLine("\nPrograma finalizado. ¡Hasta luego!");
                        break;

                    default:
                        Console.WriteLine("\nERROR: Opción no válida.");
                        Pausar();
                        break;
                }
            }
        } while (opcion != 6);
    }
}

```

```

    }
}
else
{
    Console.WriteLine("\nERROR: Debe ingresar un número.");
    Pausar();
    opcion = 0;
}

} while (opcion != 6);
}

static void AgregarTarea()
{
    Console.Clear();

    Console.WriteLine("===== AGREGAR TAREA =====");
    Console.Write("Ingrese el nombre de la tarea: ");

    string nombre = Console.ReadLine();

    if (string.IsNullOrEmpty(nombre))
    {
        Console.WriteLine("\nERROR: El nombre de la tarea no puede estar vacío.");
    }
    else
    {
        tareas.Add(new Tarea(nombre));

        Console.WriteLine("\nTarea agregada correctamente.");
    }

    Pausar();
}

static void ListarTareas()
{
    Console.Clear();

    Console.WriteLine("===== LISTA DE TAREAS =====");

    if (tareas.Count == 0)
    {
        Console.WriteLine("\nNo hay tareas registradas.");
    }
    else
    {
        for (int i = 0; i < tareas.Count; i++)
        {
            string estado;

            if (tareas[i].Completada)
            {
                estado = "COMPLETADA";
            }
            else
            {
                estado = "PENDIENTE";
            }

            Console.WriteLine(
                $"{i + 1}. {tareas[i].Nombre} - {estado}"
            );
        }
    }

    Pausar();
}

```

```

}

static void CompletarTarea()
{
    Console.Clear();

    Console.WriteLine("===== COMPLETAR TAREA =====");

    if (tareas.Count == 0)
    {
        Console.WriteLine("\nNo hay tareas registradas.");
        Pausar();
        return;
    }

    ListarTareasSinPausa();

    Console.Write("\nIngrese el número de la tarea a completar: ");

    if (int.TryParse(Console.ReadLine(), out int numero))
    {
        if (numero >= 1 && numero <= tareas.Count)
        {
            if (tareas[numero - 1].Completada)
            {
                Console.WriteLine("\nERROR: Esta tarea ya está completada.");
            }
            else
            {
                tareas[numero - 1].Completada = true;

                Console.WriteLine(
                    "\nTarea completada correctamente."
                );
            }
        }
        else
        {
            Console.WriteLine("\nERROR: El número de tarea no existe.");
        }
    }
    else
    {
        Console.WriteLine("\nERROR: Debe ingresar un número válido.");
    }

    Pausar();
}

static void EliminarTarea()
{
    Console.Clear();

    Console.WriteLine("===== ELIMINAR TAREA =====");

    if (tareas.Count == 0)
    {
        Console.WriteLine("\nNo hay tareas registradas.");
        Pausar();
        return;
    }

    ListarTareasSinPausa();

    Console.Write("\nIngrese el número de la tarea a eliminar: ");

    if (int.TryParse(Console.ReadLine(), out int numero))

```



```

{
    if (numero >= 1 && numero <= tareas.Count)
    {
        string nombre = tareas[numero - 1].Nombre;

        tareas.RemoveAt(numero - 1);

        Console.WriteLine(
            $"La tarea \"{nombre}\" fue eliminada correctamente."
        );
    }
    else
    {
        Console.WriteLine("ERROR: El número de tarea no existe.");
    }
}
else
{
    Console.WriteLine("ERROR: Debe ingresar un número válido.");
}

Pausar();
}

static void ListarTareasSinPausa()
{
    for (int i = 0; i < tareas.Count; i++)
    {
        string estado = tareas[i].Completada
            ? "COMPLETADA"
            : "PENDIENTE";

        Console.WriteLine(
            $"{i + 1}. {tareas[i].Nombre} - {estado}"
        );
    }
}

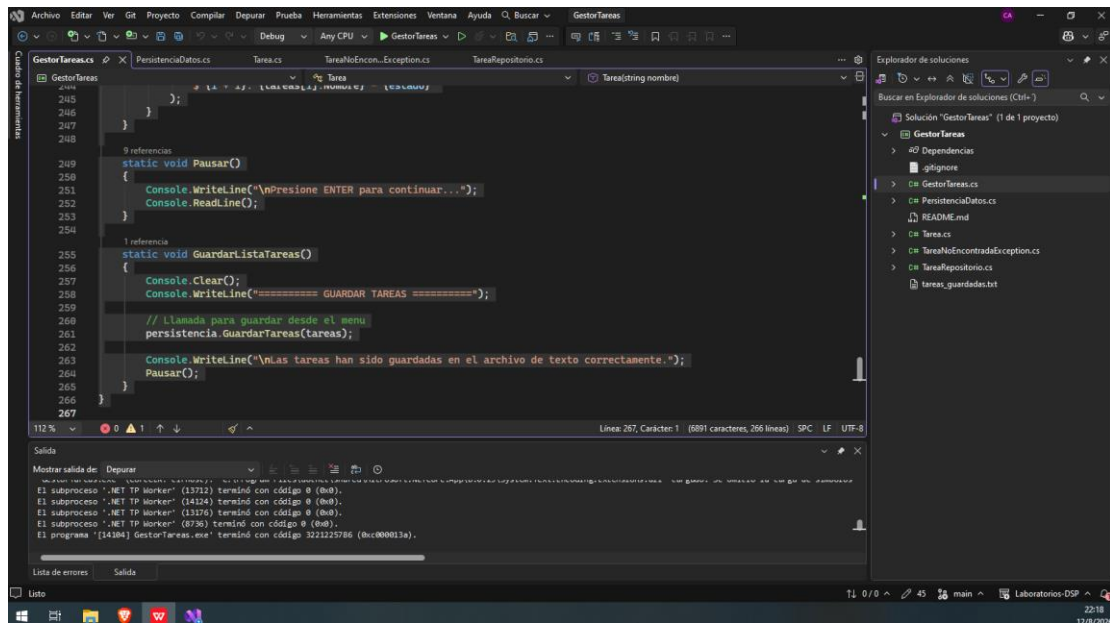
static void Pausar()
{
    Console.WriteLine("Presione ENTER para continuar...");
    Console.ReadLine();
}

static void GuardarListaTareas()
{
    Console.Clear();
    Console.WriteLine("===== GUARDAR TAREAS =====");

    // Llamada para guardar desde el menu
    persistencia.GuardarTareas(tareas);

    Console.WriteLine("Las tareas han sido guardadas en el archivo de texto correctamente.");
    Pausar();
}
}

```



Link del repositorio

https://github.com/EduardoRamirez86/InvestigacionAplicadaDSP404_consolea

C:\Users\Charlie._saa\Documents\CHARLIE\DSP\Laboratorios-DSP\TEO DSP\InvestigacionApI

```
=====
                        GESTOR DE TAREAS
=====
1. Agregar tarea
2. Listar tareas
3. Completar tarea
4. Eliminar tarea
5. Guardar lista de tareas
6. Salir
=====
Seleccione una opción: 1
```

```
C:\Users\Charlie_saa\Documents\CHARLIE\DSP\Laboratorios-DSP\TEO DSP\InvestigacionAplicadaDSP404_consola-main\bin
===== AGREGAR TAREA =====
Ingrese el nombre de la tarea: estudiar para el examen

Tarea agregada correctamente.

Presione ENTER para continuar...

```

```
C:\Users\Charlie_saa\Documents\CHARLIE\DSP\Laboratorios-DSP\TEO DSP\Investigacio
=====
GESTOR DE TAREAS
=====
1. Agregar tarea
2. Listar tareas
3. Completar tarea
4. Eliminar tarea
5. Guardar lista de tareas
6. Salir
=====
Seleccione una opción: 2

```



```
===== COMPLETAR TAREA =====
1. Demo 1 - PENDIENTE
2. Demo 3 - COMPLETADA
3. Demo 4 - PENDIENTE
4. Demo 5 - PENDIENTE
5. Demo 7 - PENDIENTE
6. estudiar para el examen - PENDIENTE
7. repasar poo y hacer guias - PENDIENTE

Ingresa el número de la tarea a completar: _
```

```
===== COMPLETAR TAREA =====
1. Demo 1 - PENDIENTE
2. Demo 3 - COMPLETADA
3. Demo 4 - PENDIENTE
4. Demo 5 - PENDIENTE
5. Demo 7 - PENDIENTE
6. estudiar para el examen - PENDIENTE
7. repasar poo y hacer guias - PENDIENTE

Ingresa el número de la tarea a completar: 7

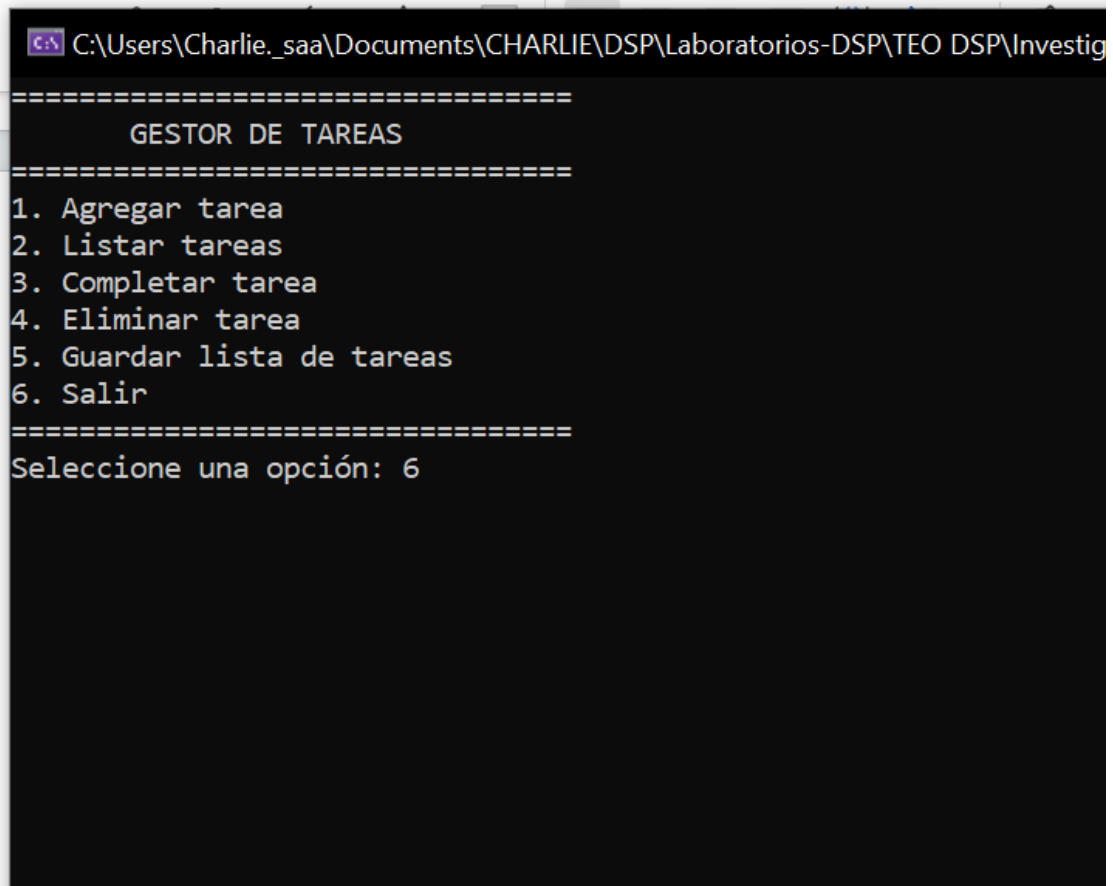
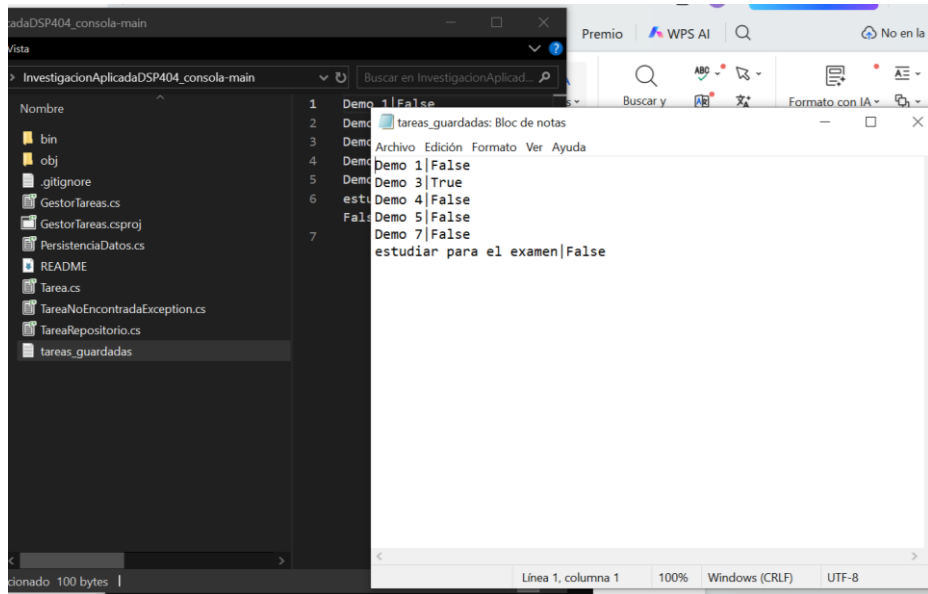
Tarea completada correctamente.

Presione ENTER para continuar...
_
```



```
C:\Users\Charlie_saa\Documents\CHARLIE\DSP\Laboratorios-DSP\TEO DSP\Inv  
===== LISTA DE TAREAS =====  
1. Demo 1 - PENDIENTE  
2. Demo 3 - COMPLETADA  
3. Demo 4 - PENDIENTE  
4. Demo 5 - PENDIENTE  
5. Demo 7 - PENDIENTE  
6. estudiar para el examen - PENDIENTE  
  
Presione ENTER para continuar...  
_
```

```
C:\Users\Charlie_saa\Documents\CHARLIE\DSP\Laboratorios-DSP\TEO DSP\InvestigacionAplicadaDSP404_conso  
===== GUARDAR TAREAS =====  
  
Las tareas han sido guardadas en el archivo de texto correctamente.  
  
Presione ENTER para continuar...  
_
```

```
poj 10 A A Aa : 2 = = A => => A
! v
Consola de depuración de Microsoft Visual Studio

=====
GESTOR DE TAREAS
=====
1. Agregar tarea
2. Listar tareas
3. Completar tarea
4. Eliminar tarea
5. Guardar lista de tareas
6. Salir
=====
Seleccione una opción: 6

Programa finalizado. ¡Hasta luego!

C:\Users\Charlie._saa\Documents\CHARLIE\DSP\Laboratorios-DSP\TEO DSP\Investigac
net8.0\GestorTareas.exe (proceso 14208) se cerró con el código 0 (0x0).
Para cerrar automáticamente la consola cuando se detiene la depuración, habilit
Cerrar la consola automáticamente al detenerse la depuración.
Presione cualquier tecla para cerrar esta ventana. . .
```